

NGCF Review

Paper: Neural Graph Collaborative Filtering

Sangho Kim

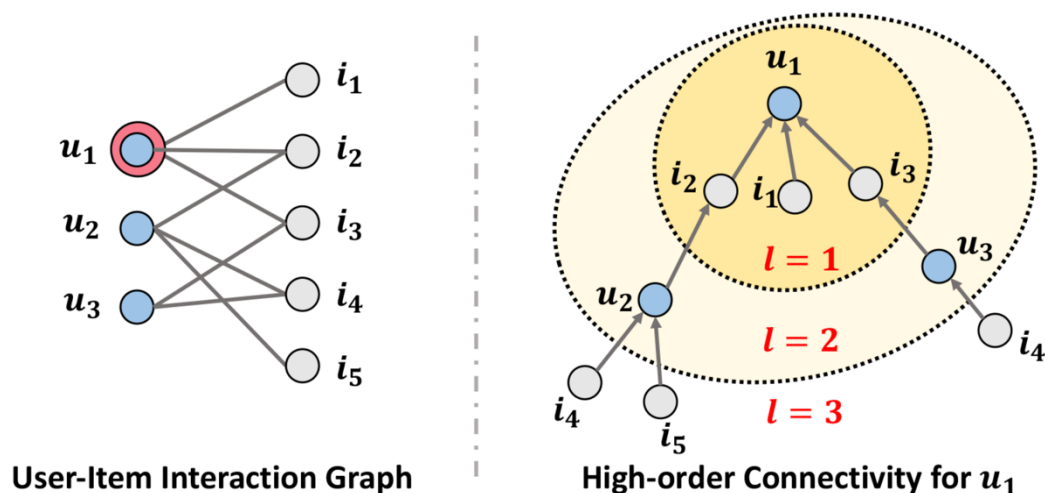
Contents

- Introduction
- Methodology
- Experiments
- Conclusion
- Future Works

Introduction

- 일반적으로 학습 가능한 CF 모델에는 2개의 핵심 요소가 존재
 - User와 Item을 Vector 형태로 표현하는 Embedding
 - 그리고 이 Embedding을 기반으로 하는 Interaction modeling
- 그러나 기존 방법론들은 후자인 Interaction modeling에 집중을 함
- 이 방법론들이 효과적이긴 하지만 Embedding 부분에서 조금 약하다고 이 논문에서 주장
- Embedding 과정에서 collaborative signal이 굉장히 중요한데 이 부분이 부족함
- 따라서 이 논문에서는 이러한 정보들을 Embedding 사이드에서 새롭게 Neural Net을 구축하는 것을 제안
- GNN에서 영감을 받았으며 우리가 제안한 layer를 여러 개 쌓을 경우, high-order connectivity 속의 collaborative signal을 포착하도록 강제할 수 있음

Introduction



- 그림으로 살펴보자
- 왼쪽 그림을 보면 단순히 u_1 과 i_1 이 interaction을 했다는 1차원적 정보만 있는 반면
- 오른쪽 그림의 경우, $l=2$ 경로의 경우 u_1 과 u_2 가 동일한 아이템 i_2 를 소비했으므로 u_1 과 u_2 는 행동 패턴이 유사한 유저임을 알 수 있음
- 또한 $l=3$ 의 경우, 유사한 유저 u_2 가 i_4 를 소비했기 때문에 u_1 역시 i_4 를 소비할 가능성이 높다고 추론 가능
- NGCF는 이러한 아이디어를 기반으로 하여 새로운 Neural Net을 제안

Contents

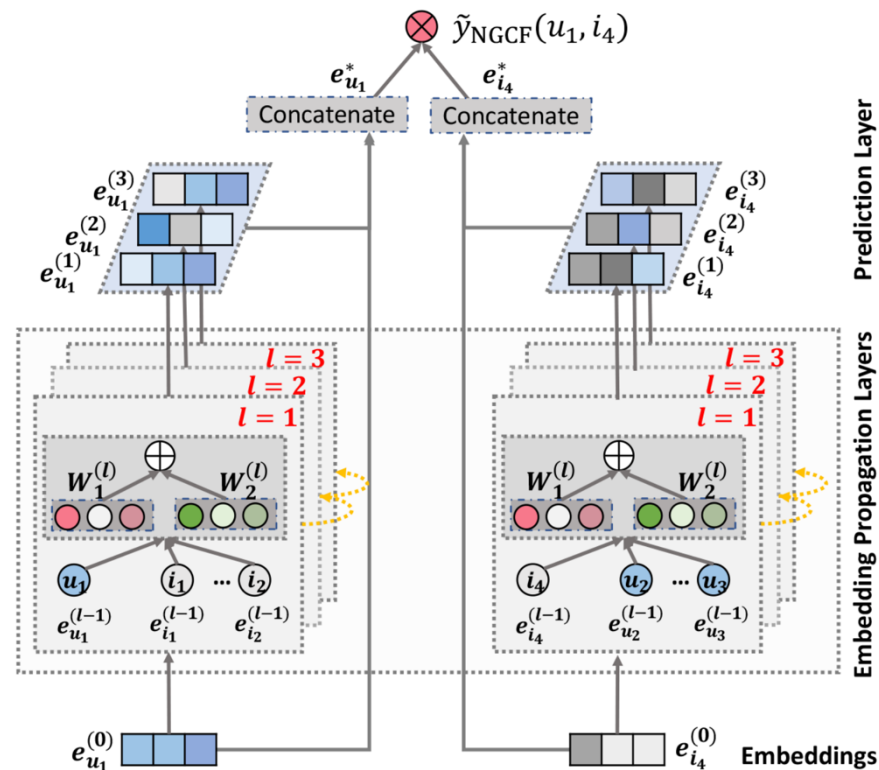
- Introduction
- **Methology**
- Experiments
- Conclusion
- Future Works

Methology

- 이 Framework에는 3개의 구성요소가 존재
- 첫 번째 Embedding Layer
 - User embedding과 Item embedding의 initialization을 제공
 - 이 친구는 단순히 두 embedding을 합쳐 **initial state로 trainable parameter 형태로 정의**를 함
- 두 번째 Embedding Propagation Layer
 - 이는 **GNN 기반으로 구축**
 - **first-order propagation**부터 보자면, 이 프로세스에서는 **item에 대한 user들의 직접적인 선호도를 표현**
 - 이를 바탕으로 embedding propagation을 수행
 - 이 과정을 message construction, message aggregation이라는 두 개의 주요 연산으로 진행
 - 자세한 수식은 논문 참고
 - 다음으로 **high-order propagation**을 보면 first-order connectivity modeling을 통해 **representation이 더 많아졌고 이러한 정보를 통해 user, item 사이의 collaborative signal을 인코딩**
 - 이 역시 자세한 수식은 논문 참고

Methology

- 앞서 서술했던 것들을 아키텍처로 표현하면 오른쪽과 같음
- 즉, 단순히 초기 벡터끼리 내적하는 MF 방법론과 달리 이 논문에서 제안한 모델은 정보를 섞는 과정을 여러 번 반복
- $l = 1$ 에서는 u_1 은 자신의 소비한 $i_1, i_2, \dots$ 의 특성을 반영하는 과정이고
- $l = 2$ 에서는 u_1 은 자신과 비슷한 다른 user들의 정보까지 반영하는 과정이고
- $l = 3$ 에서는 이보다 더 복잡한 item의 정보까지 반영
- 즉, 화살표들이 위로 올라가며 feature가 점점 풍부해짐



Methology

- 그 다음으로 Model Prediction이 어떻게 되는지 살펴보자
- l 개의 layer들을 거친 후, user에 대한 multiple representation vector들을 얻게 됨
- 이 vector들을 concatenate하여 최종 embedding vector를 구성
- Concat 말고도 weighted average, max pooling, LSTM 등을 활용할 수 있지만 여기서는 concat을 활용
- 이에 대한 장점은 추가적인 parameter가 필요 없으며 이 방법론이 매우 효과적인 것이 다른 논문에서 알려짐

$$Loss = \sum_{(u,i,j) \in O} -\ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) + \lambda \|\Theta\|_2^2$$

- 그 다음으로 위의 Loss function을 보면 BPR Loss라고 하는데 BCE Loss와 굉장히 유사하지만 의미는 살짝 다르다
- 메커니즘으로는 user가 interaction한 item은 interaction하지 않은 item보다 더 스코어를 잘 추론하도록 하는 것
- 즉, Rec Sys에서 유저가 이 아이템을 클릭했다는 정보는 있지만 클릭하지 않은 아이템도 있을텐데 이걸 싫어하는지는 확실하지 않으므로 안정적인 학습을 위해 위와 같이 Loss function을 설계

Methology

- 이 논문에서는 Over-fitting을 해결하기 위해 두 개의 Dropout 메소드를 활용
- Message Dropout & Node Dropout
- 말 그대로 Message Dropout은 Message를 랜덤으로 드랍하고 Node Dropout은 Node를 랜덤으로 드랍
- 그리고 당연하겠지만 Dropout은 training 할 때만 활성화
- 그리고 NGCF의 수식을 보면 SVD++의 구조를 찾아낼 수 있다고 함
- 단, First-order만 고려하며 Leaky ReLU와 같은 activation function을 제거하며 transformation matrix를 제거할 때 말이다
- 그러면 SVD++의 수식과 일치하며 결국 Matrix Decomposition 문제로 귀결이 되는 것을 강조하고 싶었던 것 같음
- 그리고 기존 MF 방법론들과 비교해 Training에는 약 4배, 추론에는 약 3배 이상 소요가 되는데 high-order connectivity까지 고려하기 위해서는 어쩔 수 없으며 충분히 효율적이라고 이 논문에서는 주장 중
- 실제 Production 환경에서 데이터를 넣을 때 얼마나 시간이 소요되는지 봐야 알겠지만 딱히 효율적이진 않은 것 같음

Contents

- Introduction
- Methodology
- Experiments
- Conclusion
- Future Works

Experiments

- 이제 실험 세팅을 살펴보자
- Dataset으로는 Gowalla, Yelp2018, Amazon-book 3개의 Benchmark를 활용했으며
- Metric으로는 일반적인 프로토콜 recall@K, ndcg@K**3을 채택 (K=20)
- Hyperparameter로는 Embedding size는 64로 고정, Optimizer는 Adam, Batch Size는 1024로 설정했으며 LR, L2 normalization, Dropout rate 등은 grid search로 실험 진행
- 그 외에 Early stopping, Xavier initialization method를 활용 그리고 L은 3으로 세팅

Experiments

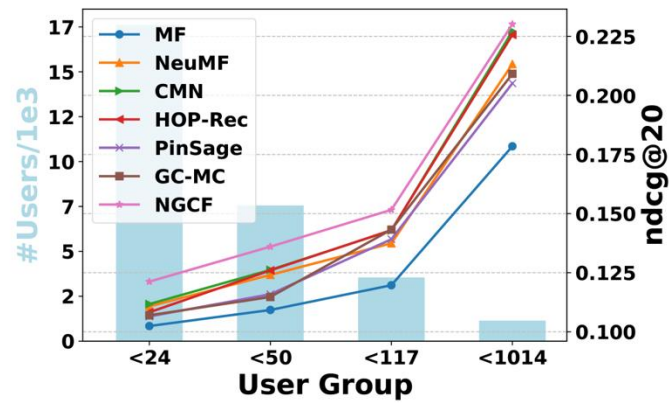
- 뻔하지만 실험 결과를 그래도 보자
- 역시 이 논문의 모델이 모든 Benchmark에서 가장 좋은 성능을 보여준다

Table 2: Overall Performance Comparison.

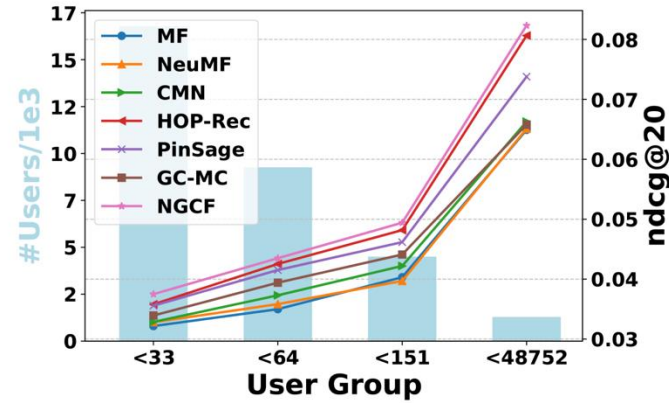
	Gowalla		Yelp2018*		Amazon-Book	
	recall	ndcg	recall	ndcg	recall	ndcg
MF	0.1291	0.1109	0.0433	0.0354	0.0250	0.0196
NeuMF	0.1399	0.1212	0.0451	0.0363	0.0258	0.0200
CMN	<u>0.1405</u>	<u>0.1221</u>	0.0457	0.0369	0.0267	0.0218
HOP-Rec	0.1399	0.1214	<u>0.0517</u>	<u>0.0428</u>	<u>0.0309</u>	<u>0.0232</u>
GC-MC	0.1395	0.1204	0.0462	0.0379	0.0288	0.0224
PinSage	0.1380	0.1196	0.0471	0.0393	0.0282	0.0219
NGCF-3	0.1569*	0.1327*	0.0579*	0.0477*	0.0337*	0.0261*
%Improv.	11.68%	8.64%	11.97%	11.29%	9.61%	12.50%
<i>p</i> -value	2.01e-7	3.03e-3	5.34e-3	4.62e-4	3.48e-5	1.26e-4

Experiments

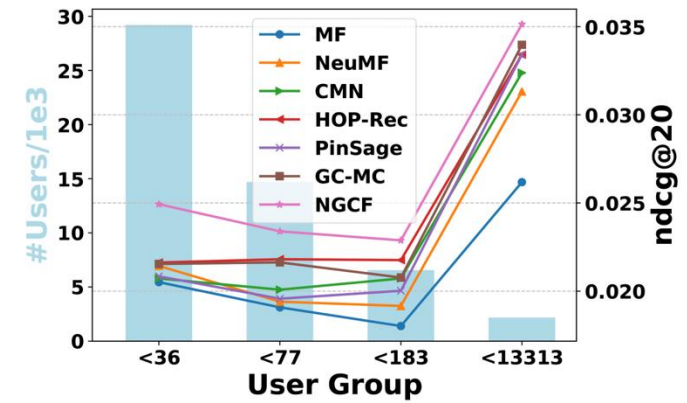
- 주목할만한 결과로는 ndcg score가 가장 높다는 점
- 즉, high-order connectivity를 고려한 설계가 먹혔다는 것이다



(a) ndcg on Gowalla



(b) ndcg on Yelp2018*



(c) ndcg on Amazon-book

Experiments

- Embedding Propagation Layer의 효과를 보여주기 위한 실험도 진행
- 아래 테이블을 보면 NGCF-1이 다른 모델보다 recall과 ndcg가 높음
- 이는 interaction term과 non-linearity를 활용한 것이 성능이 좋다는 것을 시사함

Table 4: Effect of graph convolution layers.

	Gowalla		Yelp2018*		Amazon-Book	
	recall	ndcg	recall	ndcg	recall	ndcg
NGCF-1	0.1556	0.1315	0.0543	0.0442	0.0313	0.0241
NGCF-1 _{SVD++}	0.1517	0.1265	0.0504	0.0414	0.0297	0.0232
NGCF-1 _{GC-MC}	0.1523	0.1307	0.0518	0.0420	0.0305	0.0234
NGCF-1 _{PinSage}	0.1534	0.1308	0.0516	0.0420	0.0293	0.0231

Contents

- Introduction
- Methology
- Experiments
- Conclusion
- Future Works

Conclusion

- 이 논문은 다른 방법론들과 달리 graph 구조를 Embedding function에 녹여냈다는 것이 가장 큰 임팩트
- 또한 high-order connectivity에 대한 정보를 반영하기 위한 Embedding Propagation이 굉장히 성공적

Contents

- Introduction
- Methology
- Experiments
- Conclusion
- Future Works

Future Works

- 이 논문의 저자들은 아래와 같이 Future Works를 제시
- 먼저, Attention Mechanism을 도입
 - 이를 도입함으로써 어떤 이웃 노드가 더 중요한지 모델이 스스로 판단하도록 함으로써 connectivity를 세밀하게 다듬는 것
- Adversarial Learning 활용
 - 모델의 Robustness를 높이기 위해 user/item embedding이나 graph 구조에 noise를 주고 학습시키는 Adversarial Learning 기법을 활용해보겠다고 함
- 마지막으로 구조적 정보를 다각화
 - 현재는 User-Item 정보만 활용하고 있는데 추후 Knowledge Graph, SNS, Context-aware features 등을 활용해 더 풍부한 Net을 만드는 것이 목표라고 함